

Individual Project Progress Report

(June 8 – July 15)

(Name: Fahim Foysal. Id: 2233594642)

Week 1 (Jun 8 – Jun 10)

Classes for CSE440.1 (Artificial Intelligence) had just started, and we didn't have a team yet. During this week, I reviewed the course outline and project guidelines so I would be prepared once we selected a team and topic.

Week 2 (Jun 15 – Jun 17)

We finally formed our four person team this week. We sat down together, went through a few possible project ideas which is given in the classroom, and ended up picking **Autonomous Volcanic Terrain Exploration Using a Markov Decision Process (MDP)**. The idea is a simulated rover that has to move across a randomly generated dangerous grid full of lava flows, craters, gas pockets, and rocks, while trying to explore as much as possible and collect science points without getting destroyed. An MDP made a lot of sense for this because the rover's moves don't always go as planned, and every decision only depends on where the rover currently is, which is exactly the kind of problem states, actions, rewards, and policies are meant for.

Week 3 (Jun 22 – Jun 24)

This week we set up a shared GitHub repository and split the project into four parts so everyone could work in parallel, the MDP core (terrain generation, states/actions, rewards, value iteration), the agent and simulation, visualization and demo output, and experiments/comparison. I took the **Agent & Simulation** part (Member 2). In simple terms, my job is to make the rover actually move around the terrain by following the policy that comes out of the MDP core, without changing anything inside the MDP code itself.

Week 4 (Jun 29 – Jul 1)

Instead of jumping straight into coding, I spent this week reading `support/terrain.py` and `support/mdp.py` carefully. I wanted my code to work with the real interfaces, not with my guesses about them. The most important method for me was `VolcanicMDP.transition_probabilities(state,`

`action)`, which gives back every possible next state along with its probability when the rover tries to move (75% chance it goes where it wanted, 10% it slips left, 10% it slips right, and 5% it just stays where it is). I also looked at `reward(state, action, next_state, visited)` and `extract_policy()`, which basically tells you the best action for every cell on the grid. Once I understood all that, I sketched a rough plan for `support/agent.py` (an `MDPExplorerAgent` class) and `support/simulation.py` (a `Simulation` class that connects `Terrain` → `VolcanicMDP` → `MDPExplorerAgent`). Then I created my own branch, `member2-agent-simulation`, since our team agreed that everyone works on their own branch.

Week 5 (Jul 6 – Jul 8)

This week I actually started writing `support/agent.py`. I wrote `find_base()`, which just scans the grid for the `BASE` cell so the rover always starts from the correct tile, and `choose_action()`, which looks up the current position in the MDP's policy dictionary and picks whatever action it says. The hardest part was dealing with the fact that movement is random – just because the rover picks an action doesn't mean that's what actually happens. So the agent has to randomly pick one real outcome from the probability list the MDP gives it. For this, I used my own seeded `random.Random` object instead of the global `random` module, so that every run gives the same result when the seed is the same. That will be important later when we compare the agent against baseline strategies. On top of that, I added reward tracking, a counter for how many times the rover enters a hazard, science-point tracking, and an `alive` flag that turns `False` the moment the rover steps into a LAVA cell.

Week 6 (Jul 13 – Jul 15) – Update 1 Complete

• Finished `MDPExplorerAgent`. It now

keeps track of position, path, visited, cumulative_reward, hazards_entered, science_points_collected, and alive, and mission_summary() puts everything together into one dictionary.

- Finished Simulation. It builds the VolcanicMDP, runs value_iteration() and extract_policy(), and then keeps calling agent.step() until one of three things happens: max_steps is reached, the rover dies, or it manages to cover at least 60% of the reachable cells.
- Added calculate_coverage() and a common summary-dictionary format (total_steps, cumulative_reward, coverage_percent, hazards_entered, science_points_collected, survived, path_length, final_position). This way Member 3 (visualization) and Member 4 (experiments) can directly use my output without writing any extra glue code.
- Made sure both acceptance checks – python support/agent.py and python support/simulation.py – run from start to finish with no errors.
- Committed and pushed everything to GitHub on my branch member2-agent-simulation, with the message “Add MDP agent and simulation loop.”
- Did not touch terrain.py or mdp.py, keeping to our team’s file-ownership rule.

Testing and Verification

Before pushing anything, I ran both acceptance scripts from the project root to make sure each module runs all the way through and prints the expected output:

```
$ python support/agent.py
Starting position (BASE): (9, 0)
Steps taken: 20
Cumulative reward: 290.00
Hazards entered: 1
Science points collected: 1
Survived: True
Cells visited: 5
Final position: (9, 4)
```

```
$ python support/simulation.py
Total steps: 100
Cumulative reward: 1730.00
Coverage: 6.1%
Hazards entered: 1
Science points collected: 1
Survived: True
Path length: 101
Final position: (9, 4)
```

Both runs finished without any exceptions. The rover started exactly at BASE, followed the MDP policy through the terrain even with the random movement, and the simulation stopped at max_steps because 100 steps weren’t enough to reach the 60% coverage target. That’s exactly how it’s supposed to stop, it wasn’t stuck in some infinite loop.

My Personal Contribution (Summary)

Before writing any code, I made sure I properly understood the terrain and MDP code that my part depends on. Then I built MDPExplorerAgent in support/agent.py, which handles finding the base, choosing actions from the policy, sampling the random movement outcomes, and tracking everything about the mission (path, visited cells, reward, hazards, science points, and whether the rover is still alive). After that, I built Simulation in support/simulation.py, which connects Terrain, VolcanicMDP, and the agent into a single run loop with proper stopping rules (max steps, rover death, or enough coverage), along with a shared summary dictionary that the rest of the team can build on. I tested both modules to confirm they run on their own and print clear mission summaries. Finally, I committed and pushed all of it on my own branch, member2-agent-simulation, without touching anyone else’s files.